Kubernetes Cluster Autoscaler in Amazon EKS

Overview

Cluster Autoscaler automatically adjusts the number of worker nodes in an Amazon EKS cluster based on pod resource requirements.

When pods are unable to schedule due to insufficient resources, Cluster Autoscaler increases the number of worker nodes.

When nodes are underutilized, Cluster Autoscaler can remove unnecessary worker nodes to optimize infrastructure cost.

Step 1: Enable IAM Permissions for Auto Scaling Group Interaction

Create IAM Policy for Worker Node Role

After creating the EKS cluster, Cluster Autoscaler requires specific IAM permissions to interact with AWS Auto Scaling Groups.

These permissions allow Cluster Autoscaler to:

- Discover Auto Scaling Groups
- Increase worker node count
- Decrease worker node count
- Read launch template information
- Interact with EKS node groups

Steps

1. Open the AWS Management Console.
2. Navigate to IAM.
3. Select Roles.
4. Choose the Worker Node IAM Role attached to the EKS worker nodes.
5. Click Add Permissions.
6. Select Create Inline Policy.
7. Choose the JSON tab.
8. Paste the following policy.

9. Review and save the policy.

IAM Policy

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "autoscaling:DescribeAutoScalingGroups",
                "autoscaling:DescribeAutoScalingInstances",
                "autoscaling:DescribeLaunchConfigurations",
                "autoscaling:DescribeTags",
                "autoscaling:SetDesiredCapacity",
                "autoscaling:TerminateInstanceInAutoScalingGroup",
                "ec2:DescribeLaunchTemplateVersions",
                "eks:DescribeNodegroup"
            ],
            "Resource": "*"
        }
    ]
}
```

Step 2: Configure Worker Node Auto Scaling Group

Verify Auto Scaling Group Configuration

1. Open the AWS Console.
2. Navigate to EC2.
3. Select Auto Scaling Groups.
4. Locate the Auto Scaling Group created for the EKS worker nodes.

Check Minimum and Maximum Capacity

Verify the following:

- Minimum capacity
- Desired capacity
- Maximum capacity

Example:

```
Min Size      = 2
Desired Size  = 2
Max Size      = 5
```

Important Note

If:

```
Desired Capacity = 2
Max Capacity     = 2
```

Then Cluster Autoscaler cannot add new worker nodes because the maximum limit has already been reached.

To allow automatic scaling, increase the maximum node count.

Example:

```
Min Size = 2
Max Size = 5
```

Step 3: Deploy the Cluster Autoscaler

Deploy Cluster Autoscaler into the EKS cluster.

Run the following command:

```
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/autoscaler/master/cluster-
autoscaler/cloudprovider/aws/examples/cluster-autoscaler-autodiscover.yaml
```

This creates:

- Service Account
- RBAC Permissions
- Deployment
- ClusterRole
- ClusterRoleBinding

Step 4: Add Safe-To-Evict Annotation

Add the annotation to prevent Cluster Autoscaler pods from being evicted.

Run the following command:

```
kubectl -n kube-system annotate deployment.apps/cluster-autoscaler cluster-autoscaler.kubernetes.io/safe-to-evict="false"
```

Step 5: Edit the Cluster Autoscaler Deployment

Edit the deployment configuration.

Run:

```
kubectl -n kube-system edit deployment.apps/cluster-autoscaler
```

Update the Deployment Arguments

Inside the deployment configuration:

Replace:

```
<YOUR CLUSTER NAME>
```

with your actual EKS cluster name.

Add the Following Additional Arguments

```
--balance-similar-node-groups
--skip-nodes-with-system-pods=false
```

Example Container Arguments

```
containers:
- command:
  - ./cluster-autoscaler
  - --v=4
  - --stderrthreshold=info
  - --cloud-provider=aws
```

```
  - --skip-nodes-with-local-storage=false
  - --expander=least-waste
  - --node-group-auto-discovery=asg:tag=k8s.io/cluster-
autoscaler/enabled,k8s.io/cluster-autoscaler/<YOUR_CLUSTER_NAME>
  - --balance-similar-node-groups
  - --skip-nodes-with-system-pods=false
```

Save the Deployment

To save and exit:

```
Press ESC
Type :wq
Press Enter
```

Kubernetes automatically updates the deployment.

Step 6: Update the Cluster Autoscaler Image Version

The Cluster Autoscaler image version must match the Kubernetes version used in EKS.

Example:

If the EKS Kubernetes version is:

```
1.30
```

Use the corresponding Cluster Autoscaler image:

```
v1.30.1
```

Update the Image

Run the following command:

```
kubectl set image deployment cluster-autoscaler -n kube-system cluster-
autoscaler=registry.k8s.io/autoscaling/cluster-autoscaler:v1.30.1
```

Step 7: Verify Cluster Autoscaler Pods

Check whether the Cluster Autoscaler pod is running.

```
kubectl get pods -n kube-system | grep cluster-autoscaler
```

Expected output:

```
cluster-autoscaler-xxxxxxxxxx-xxxxx   1/1   Running
```

Step 8: Check Cluster Autoscaler Logs

View logs for troubleshooting and validation.

```
kubectl logs -n kube-system deployment.apps/cluster-autoscaler
```

Verify Cluster Autoscaler is Working

Look for messages similar to:

```
Scale-up: setting group desired size
```

or

```
Successfully increased node group size
```

These messages indicate successful node scaling.

How Cluster Autoscaler Works

Scale-Up Process

Cluster Autoscaler increases worker nodes when:

- Pods remain in Pending state
- Insufficient CPU or memory exists
- Existing nodes cannot schedule new pods

Scale-Down Process

Cluster Autoscaler removes worker nodes when:

- Nodes are underutilized
- Pods can be moved safely to other nodes
- Nodes remain unused for a configured duration

Verify Worker Nodes

Check worker nodes:

`kubectl get nodes`

Monitor whether new nodes are added automatically during increased workload.

Generate Load for Testing

Example deployment to generate CPU load:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cpu-stress

spec:
  replicas: 20

  selector:
    matchLabels:
      app: cpu-stress

  template:
    metadata:
      labels:
        app: cpu-stress

    spec:
      containers:
      - name: stress
        image: busybox
        command:
        - /bin/sh
        - -c
```

```
        - while true; do echo hello; sleep 10; done

        resources:
          requests:
            cpu: "500m"
            memory: "256Mi"
```

Apply the deployment:

```
kubectl apply -f cpu-stress.yaml
```

If insufficient nodes exist, Cluster Autoscaler automatically provisions additional worker nodes.

Useful Commands

Check Nodes

```
kubectl get nodes
```

Check Pending Pods

```
kubectl get pods --all-namespaces | grep Pending
```

Check Cluster Autoscaler Deployment

```
kubectl get deployment cluster-autoscaler -n kube-system
```

Describe Cluster Autoscaler Pod

```
kubectl describe pod -n kube-system <cluster-autoscaler-pod-name>
```

Restart Cluster Autoscaler

```
kubectl rollout restart deployment cluster-autoscaler -n kube-system
```

Best Practices

Match Autoscaler Version with Kubernetes Version

Always use a compatible Cluster Autoscaler version.

Example:

Kubernetes Version	Cluster Autoscaler Version

Kubernetes Version	Cluster Autoscaler Version
1.28	v1.28.x
1.29	v1.29.x
1.30	v1.30.x

Enable Proper Resource Requests

Cluster Autoscaler works based on pod resource requests.

Always define:

```
resources:
  requests:
    cpu:
    memory:
```

Without requests, autoscaler decisions may not work correctly.

Recommended Architecture

Component	Purpose
HPA	Scales pod count
VPA	Optimizes pod resources
Cluster Autoscaler	Scales worker nodes

Cleanup

Delete Cluster Autoscaler

```
kubectl delete -f
https://raw.githubusercontent.com/kubernetes/autoscaler/master/cluster-
autoscaler/cloudprovider/aws/examples/cluster-autoscaler-autodiscover.yaml
```

Summary

Cluster Autoscaler in EKS:

- Automatically adds worker nodes during high load

- Removes unused worker nodes during low utilization
- Optimizes infrastructure usage and cost
- Works together with HPA and VPA for complete Kubernetes autoscaling